On Implementing Stabilizing Leader Election with Weak Assumptions on Network Dynamics

Karine Altisen
Karine.Altisen@univ-grenoble-alpes.fr
Université Grenoble Alpes, VERIMAG, UMR 5104
Grenoble, France

Stéphane Devismes
Stephane.Devismes@univ-grenoble-alpes.fr
Université Grenoble Alpes, VERIMAG, UMR 5104
Grenoble, France

Anaïs Durand
Anais.Durand@uca.fr
LIMOS, Université Clermont Auvergne, CNRS, UMR 6158
Clermond-Ferrand, France

Colette Johnen
Colette.Johnen@lip6.fr
Sorbonne Université, LIP6, CNRS, INRIA
Paris, France

Franck Petit
Franck.Petit@lip6.fr
Sorbonne Université, LIP6, CNRS, INRIA
Paris, France

ABSTRACT

We consider self-stabilization and its weakened form called pseudo-stabilization. We study conditions under which (pseudo- and self-) stabilizing leader election is solvable in networks subject to frequent topological changes. To model such an high dynamics, we use the dynamic graph (DG) paradigm and study a taxonomy of nine important DG classes. Our results show that self-stabilizing leader election can only be achieved in the classes where all processes are sources. Furthermore, even pseudo-stabilizing leader election cannot be solved in all remaining classes, except in the class where at least one process is a timely source. We illustrate that result by proposing a pseudo-stabilizing leader election algorithm for the latter class. We also show that in this last case, the convergence time of pseudo-stabilizing leader election algorithms cannot be bounded. Nevertheless, we show that our solution is speculative since its convergence time can be bounded when the dynamics is not too erratic, precisely when all processes are timely sources.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

Self-stabilization, pseudo-stabilization, dynamic graphs, leader election, speculation.

ACM Reference Format:
Karine Altisen, Stéphane Devismes, Anaïs Durand, Colette Johnen, and Franck Petit. 2021. On Implementing Stabilizing Leader Election with Weak Assumptions on Network Dynamics. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC '21), July 26–30, 2021, Virtual Event, Italy.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3465084.3467917

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '21, July 26–30, 2021, Virtual Event, Italy.
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8548-0/21/07…$15.00
https://doi.org/10.1145/3465084.3467917

The full version of this paper is available at https://hal.archives-ouvertes.fr/hal-02979166.

1 INTRODUCTION

Modern networks (*e.g.*, MANET, VANET, and DTN) are prone to both *faults* and *frequent alteration of their topology* (*i.e.*, the addition or removal of links). Several works aim at capturing such a network dynamics using graph-based models. In [7, 11], the network dynamics is represented as a sequence of digraphs called *evolving or dynamic graphs*. In [10], the topological evolution of the network is modeled by a *Time-Varying Graph* (TVG). A TVG consists of a (fixed) digraph and a presence function which indicates whether or not a given arc of the digraph exists at a given time.

The way a network is connected over time is of prime interest to develop algorithmic solutions. On the other hand, an algorithm should be as general as possible and so not focus on a single dynamic pattern. Therefore, dynamic patterns are often classified according to the temporal characteristics of edge presence they satisfy; see, *e.g.*, [10]. Such taxonomies are always proposed together with possibility/impossibility results related to the strength of classes that compose them.

In [2], the authors have initiated research to unify *fault-tolerance* and *endurance to high-frequency of topological changes* by proposing self-stabilizing algorithms for the leader election problem in three important classes of dynamic networks, modeled using the TVG paradigm. In the present paper, we explore conditions on the network dynamics under which the (deterministic) stabilizing leader election is solvable. The term "stabilization" covers here both *self-stabilization* [14] and its weakened deterministic variant called *pseudo-stabilization* [8] (*n.b.*, by definition, any self-stabilizing algorithm is pseudo-stabilizing, but the reverse is not necessarily true). Essentially, these two properties express the ability of a distributed algorithm to withstand transient faults, *i.e.*, faults (such as memory corruption) that occur at an unpredictable time, but do not result in a permanent hardware damage and whose frequency is low. Indeed, starting from an arbitrary configuration (which may be the

result of transient faults), a self-stabilizing algorithm makes a distributed system reach within finite time a configuration from which *all possible execution suffixes* satisfy the intended specification. In contrast, an algorithm is pseudo-stabilizing if all its executions, starting from arbitrary configurations, have *a suffix* satisfying the intended specification. Up to now, these two stabilizing properties have been mainly studied in static systems, *i.e.*, assuming a network with a fixed topology. In highly dynamic networks, the design of stabilizing solutions is challenging since the network topology continuously evolves over the time. Notably, stabilization should be achieved in spite of the high-frequency of topological changes occurring throughout the convergence.

As shown by this paper, pseudo- (and so self-) stabilization is often simply impossible to achieve when the dynamics is too erratic. Furthermore, when stabilization is possible, the convergence time of stabilizing solutions is often unboundable. In such cases, *speculation* [20] makes sense. Indeed, an algorithm is speculative whenever it satisfies its requirements for all executions, but also exhibits significantly better performances in a subset of more probable executions. Speculative self-stabilization has been initially investigated in static networks [19]. Yet, recently, speculative self-stabilizing solutions for dynamic networks have been proposed [2]: when convergence time cannot be bounded in a very general class, an important subclass where it can be is exhibited.

Contribution. We study conditions under which stabilizing leader election can be solved in highly dynamic identified message passing systems. We model the network dynamics using the dynamic graph (DG) paradigm [11]. Moreover, we assume that processes can synchronously communicate using local broadcast primitives: at each round, every process can send a common message to its unknown set of current neighbors (if any).

Due to the intrinsic absence of termination detection, stabilizing leader election is close to the *eventual leader election* problem Ω [13] studied in crash-prone partially synchronous, yet static and fully connected, systems. In this problem, all correct processes should eventually always designate the same correct process as leader [1]. We propose here to study the stabilizing leader election problem in DG classes defined by analogy with the classes of partially synchronous systems investigated for Ω. In those latter classes, the notions of source, sink, and timeliness are central. We first accommodate these concepts to the DG paradigm. Roughly speaking, in a DG, a *source* is a process which is infinitely often able to reach all other ones by flooding. It is a *quasi-timely source* if, using flooding, it can infinitely often reach all other processes within some bounded time Δ. Finally, a *timely source* is a source that can always reach (still by flooding) all other processes within some bounded time Δ. Conversely, a *sink* is a process which is infinitely often reachable by all other ones using flooding. *Quasi-timely* and *timely sinks* are defined similarly to the quasi-timely and timely sources. Using these notions we consider a taxonomy of nine DG classes—the hierarchy among all classes of this taxonomy is presented in Figure 2.1, page 3:

- $\mathcal{J}_{1,*}$, $\mathcal{J}^{Q}_{1,*}(\Delta)$, and $\mathcal{J}^{B}_{1,*}(\Delta)$ contain every DG where at least one (*a priori* unknown) process is a source, a quasi-timely source, and a timely source, respectively.
- $\mathcal{J}_{*,1}$, $\mathcal{J}^{Q}_{*,1}(\Delta)$, and $\mathcal{J}^{B}_{*,1}(\Delta)$ contain every DG where at least one (*a priori* unknown) process is a sink, a quasi-timely sink, and a timely sink, respectively.
- $\mathcal{J}_{*,*}$, $\mathcal{J}^{Q}_{*,*}(\Delta)$, and $\mathcal{J}^{B}_{*,*}(\Delta)$ (the three classes studied in [2]) contain every DG where all processes are sources, quasi-timely sources, and timely sources, respectively.

The ability to solve a global task such as leader election in a given system inherently depends on the ability to broadcast information. So, it seems natural to study systems with sources achieving various timeliness guarantees (*e.g.*, simple sources, quasi-timely sources, and timely sources). Furthermore, the fact that sinks are the obvious dual of sources is the main motivation to study systems containing some of them. Then, encompassing worst-case executions, where for example there is a unique a source, may seem to be irrelevant in practice. However, the fact that a system stabilizes to a unique leader even under such weak conditions makes it more resilient. Indeed, to make the election fail, the system should suffer from numerous and frequent erratic topological changes in order to violate these dynamic assumptions. Moreover, after the end of such a disastrous period, the system will likely elect a leader again within a short period.

It is in this spirit that we study the possibility of designing stabilizing leader election in those classes. From [2], we already know that self-stabilizing leader election can be achieved in $\mathcal{J}_{*,*}$, $\mathcal{J}^{Q}_{*,*}(\Delta)$, and $\mathcal{J}^{B}_{*,*}(\Delta)$. However, in this paper, we show that those three classes are actually the only ones of our taxonomy where it is possible. Furthermore, we establish that in the remaining classes, even pseudo-stabilizing leader election is not possible, except in the class $\mathcal{J}^{B}_{1,*}(\Delta)$, *i.e.*, any DG with at least one timely source. For this latter class, we propose a pseudo-stabilizing leader election algorithm, called Algorithm $\mathcal{LE}$. We also show that, in general, convergence time of pseudo-stabilizing leader election algorithms cannot be bounded in $\mathcal{J}^{B}_{1,*}(\Delta)$. Nevertheless, we show that Algorithm $\mathcal{LE}$ is speculative in the sense that its convergence time in $\mathcal{J}^{B}_{*,*}(\Delta) \subset \mathcal{J}^{B}_{1,*}(\Delta)$ is at most $6\Delta + 2$ rounds.

Related Work. Ensuring convergence in highly dynamic networks regardless of the initial configuration is challenging, even impossible in many cases [6]. However, perhaps surprisingly, a number of self-stabilizing work, *e.g.*, [12, 15, 18] (*n.b.*, [12] deals with leader election) are advertised as solutions suited for dynamic networks. Actually, those works propose self-stabilizing algorithms dedicated to arbitrary network topologies and do not propose any specific patch to handle topological changes. Consequently, they tolerate topological changes only if they are eventually detected locally at involved processes and if the frequency of such events is low enough. Indeed, in such a case, topological changes can be seen as transient faults. Furthermore, several approaches derived from self-stabilization, *e.g.*, *superstabilization* [17] and *gradual stabilization* [3], aims at additionally providing countermeasures to efficiently treat topological changes when they are both spatially and timely sparse (*i.e.*, transient). Overall, all these approaches become totally ineffective when the frequency of topological changes drastically increases.

To the best of our knowledge, only a few self-stabilizing work [2, 5, 9, 16] deal with a highly dynamic context. In [2], the author propose three self-stabilizing algorithms that solve the leader election

problem, one for each class $\mathcal{J}_{*,*}$, $\mathcal{J}^Q_{*,*}(\Delta)$, and $\mathcal{J}^B_{*,*}(\Delta)$. Except [2], all other aforementioned works, *i.e.*, [5, 9, 16], use widely different models and assumptions than ours. In [5], authors consider the self-stabilizing exploration of a *highly dynamic ring* by a cohort of synchronous robots equipped with visibility sensors, moving actuators, yet no communication capabilities. Contrary to [5], our classes never enforce the network to have a particular topology at a given time. In [9], Cai *et al.* tackles the self-stabilizing leader election problem in highly dynamic systems through the population protocol model. In this model, communications are achieved by atomic rendezvous between pairs of anonymous processes, where ties are nondeterministically broken. The local broadcast primitive we use here is weaker. Moreover, authors assume global fairness, meaning that every configuration which is infinitely often reachable is actually infinitely often reached. We do not make such an assumption here. Finally, Dolev *et al.* in [16] assume that the system is equipped with a routing algorithm which allows any two processes to communicate, provided that the sender knows the identifier of the receiver. This black-box protocol abstracts the dynamics of the system: the dynamics makes it fair lossy, non-FIFO, and duplication-prone. Moreover, the channel capacity is assumed to be bounded. Based on this weak routing algorithm, they build a stronger routing protocol which is reliable, FIFO, and which prevents duplication. Again, the communication primitive we assume here is drastically weaker. As a matter of facts, in several classes studied here all-to-all communication is simply impossible.

Roadmap. In Section 2, we present our taxonomy of DG classes and the computational model. In the same section, we define the two stabilizing variants we will consider as well as the leader election problem. Section 3 is dedicated to our impossibility results and lower bounds. Our speculative pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$ is proposed in Section 4. We make concluding remarks in Section 5. Due to the lack of space, many technical results have been omitted; see the full version online for details (https://hal.archives-ouvertes.fr/hal-02979166).

2 PRELIMINARIES

2.1 Dynamic Graphs [11]

For any digraph G, we denote by $V(G)$ its vertex set and by $E(G)$ its set of oriented edges. A *dynamic graph $\mathcal{G}$ with vertex set V* (DG for short) is an infinite sequence of loopless digraphs $G_1, G_2, \ldots$ such that $V(G_i) = V$, for every $i \in \mathbb{N}^*$. For every $i \in \mathbb{N}^*$, we denote by $\mathcal{G}_{i\triangleright}$ the dynamic graph $G_i, G_{i+1}, \ldots$ with vertex set V, *i.e.*, the suffix of $\mathcal{G}$ starting from position i.

A *journey* $\mathcal{J}$ can be thought as a path over time from a starting vertex p_1 to a destination vertex q_k, *i.e.*, $\mathcal{J}$ is a finite non-empty sequence of pairs $\mathcal{J} = (e_1, t_1), (e_2, t_2), \ldots, (e_k, t_k)$ where $\forall i \in \{1, \ldots, k\}$, $e_i = (p_i, q_i) \in E(G_{t_i})$ and $i < k \Rightarrow q_i = p_{i+1} \wedge t_i < t_{i+1}$. We respectively denote by $departure(\mathcal{J})$ and $arrival(\mathcal{J})$ the *starting time* t_1 and the *arrival time* t_k of $\mathcal{J}$. The *temporal length* of $\mathcal{J}$ is equal to $arrival(\mathcal{J}) - departure(\mathcal{J}) + 1$. We denote by $\mathcal{J}_{\mathcal{G}}(p, q)$ the set of journeys from p to q in $\mathcal{G}$. We let $\overset{\mathcal{G}}{\rightsquigarrow}$ be the binary relation over V such that $p \overset{\mathcal{G}}{\rightsquigarrow} q$ if and only if $p = q$ or there exists a journey from p to q in $\mathcal{G}$. The *temporal distance* from p to q in $\mathcal{G}$, $\hat{d}_{\mathcal{G}}(p, q)$, is defined as follows: $\hat{d}_{\mathcal{G}}(p, q) = 0$ if $p = q$, $\hat{d}_{\mathcal{G}}(p, q) = \min\{arrival(\mathcal{J}) : \mathcal{J} \in \mathcal{J}_{\mathcal{G}}(p, q)\}$ otherwise (by convention, we let $\min \emptyset = +\infty$). Roughly speaking, the temporal distance from p to q in $\mathcal{G}$ gives the minimum timespan for p to reach q in $\mathcal{G}$. For all previous notations, we omit the subscript $\mathcal{G}$ when it is clear from the context.

A class of dynamic graphs is defined as a particular set of dynamic graphs. A DG class C is said to be *recurring* if and only if for every DG $\mathcal{G} \in C$, we have $\forall i \in \mathbb{N}^*$, $\mathcal{G}_{i\triangleright} \in C$; in other words, every recurring DG class is suffix-closed. We consider the nine classes of recurring DGs defined in Table 1.

By definition, for every $\Delta \in \mathbb{N}^*$, we have the inclusions depicted in Figure 2.1. Furthermore, Theorem 2.1 states that there is no inclusion between any other pair of classes. In particular, this means that inclusions in Figure 2.1 are strict; see the full paper online for a detailed proof.

Theorem 2.1. *$\forall \Delta \in \mathbb{N}^*$, the inclusions given in Figure 2.1 hold. These inclusions are strict and there is no inclusion between any other pair of classes.*

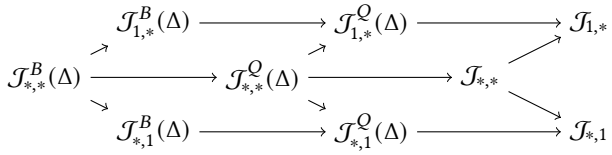


Figure 2.1: The hierarchy of studied DG classes. $A \longrightarrow B$ means that $A \subset B$.

2.2 Computational Model

We consider the computational model defined in [4, 11]. We assume a *distributed system* made of a set V of n *processes*. Each process has a local memory, a local sequential and deterministic algorithm, and message exchange capabilities. We assume that each process p has a unique identifier (ID for short).The ID of p is denoted by $id(p)$ and taken in an arbitrary domain $IDSET$ totally ordered by $<$. Processes are assumed to communicate by message passing through an interconnected network that evolves over the time. The dynamic topology of the network is then conveniently modeled by a dynamic graph $\mathcal{G} = G_1, G_2, \ldots$ with vertex set V, *i.e.*, the set of processes. Processes execute their local algorithms in *synchronous rounds*. For every $i \in \mathbb{N}^*$, the communication network at *Round i* is defined by G_i, *i.e.*, the graph at position i in $\mathcal{G}$. $\forall p \in V$, $\forall i \in \mathbb{N}^*$, we denote by $\mathcal{IN}(p)^i = \{q \in V : (q, p) \in E(G_i)\}$ the set of p's incoming neighbors at Round i. $\mathcal{IN}(p)^i$ is assumed to be unknown by p, whatever be the value of i.

A *distributed algorithm* $\mathcal{A}$ is a collection of n local algorithms $\mathcal{A}(p)$, one per process $p \in V$ (*n.b.*, different processes may have different codes). At each round, the *state* of each process $p \in V$ in $\mathcal{A}$ is defined by the values of its variables in $\mathcal{A}(p)$. Some variables may be constant in which case their values are predefined. Since any algorithm $\mathcal{A}$ is *designed for a given class of dynamic graphs* C, we assume the following property of *well-formedness*: for all $\mathcal{G}, \mathcal{G}' \in C$ with vertex sets V and V' respectively, if $|V| = |V'|$, then every process $p \in V \cap V'$ executes the same local program $\mathcal{A}(p)$ (in

Classes with a source

Class $\mathcal{J}_{1,*}$	At least one vertex, called a *(temporal) source*, can reach all the other vertices infinitely often through a journey: $\mathcal{G} \in \mathcal{J}_{1,*}$ iff $\exists src \in V, \forall p \in V, \forall i \in \mathbb{N}^*, src \overset{\mathcal{G}_{i\triangleright}}{\rightsquigarrow} p$
Class $\mathcal{J}^Q_{1,*}(\Delta)$	At least one vertex, called a *quasi-timely source*, is infinitely often at temporal distance at most Δ from each other vertex: $\mathcal{G} \in \mathcal{J}^Q_{1,*}(\Delta)$ iff $\exists src \in V, \forall p \in V, \forall i \in \mathbb{N}^*, \exists j \geq i, \hat{d}_{\mathcal{G}_{j\triangleright}}(src, p) \leq \Delta$
Class $\mathcal{J}^B_{1,*}(\Delta)$	At least one vertex, called a *timely source*, is always at temporal distance at most Δ from all other vertices: $\mathcal{G} \in \mathcal{J}^B_{1,*}(\Delta)$ iff $\exists src \in V, \forall p \in V, \forall i \in \mathbb{N}^*, \hat{d}_{\mathcal{G}_{i\triangleright}}(src, p) \leq \Delta$

Classes with a sink

Class $\mathcal{J}_{*,1}$	At least one vertex, called a *(temporal) sink*, can be reached by all the other vertices infinitely often through a journey: $\mathcal{G} \in \mathcal{J}_{*,1}$ iff $\exists snk \in V, \forall p \in V, \forall i \in \mathbb{N}^*, p \overset{\mathcal{G}_{i\triangleright}}{\rightsquigarrow} snk$
Class $\mathcal{J}^Q_{*,1}(\Delta)$	Every vertex is infinitely often at temporal distance at most Δ from at least one given vertex, called a *quasi-timely sink*: $\mathcal{G} \in \mathcal{J}^Q_{*,1}(\Delta)$ iff $\exists snk \in V, \forall p \in V, \forall i \in \mathbb{N}^*, \exists j \geq i, \hat{d}_{\mathcal{G}_{j\triangleright}}(p, snk) \leq \Delta$
Class $\mathcal{J}^B_{*,1}(\Delta)$	Every vertex is always at temporal distance at most Δ of at least one given vertex, called a *timely sink*: $\mathcal{G} \in \mathcal{J}^B_{*,1}(\Delta)$ iff $\exists snk \in V, \forall p \in V, \forall i \in \mathbb{N}^*, \hat{d}_{\mathcal{G}_{i\triangleright}}(p, snk) \leq \Delta$

Classes where every vertex is both a sink and a source

Class $\mathcal{J}_{*,*}$	Every vertex can always reach all the others through a journey: $\mathcal{G} \in \mathcal{J}_{*,*}$ iff $\forall p \in V, \forall q \in V, \forall i \in \mathbb{N}^*, p \overset{\mathcal{G}_{i\triangleright}}{\rightsquigarrow} q$
Class $\mathcal{J}^Q_{*,*}(\Delta)$	Every vertex is infinitely often at temporal distance at most Δ from each other vertex: $\mathcal{G} \in \mathcal{J}^Q_{*,*}(\Delta)$ iff $\forall p, q \in V, \forall i \in \mathbb{N}^*, \exists j \geq i, \hat{d}_{\mathcal{G}_{j\triangleright}}(p, q) \leq \Delta$
Class $\mathcal{J}^B_{*,*}(\Delta)$	Every vertex is always at temporal distance at most Δ from all other vertices: $\mathcal{G} \in \mathcal{J}^B_{*,*}(\Delta)$ iff $\forall p \in V, \forall q \in V, \forall i \in \mathbb{N}^*, \hat{d}_{\mathcal{G}_{i\triangleright}}(p, q) \leq \Delta$

Table 1: The studied recurring DG classes ($\Delta \in \mathbb{N}^*$ and $\mathcal{G}$ is a dynamic graph with vertex set V).

particular, with the same set of local states) whether p is running in $\mathcal{G}$ or $\mathcal{G}'$. This property claims that an algorithm depends only on (1) characteristics that are global to the class in which it runs (*e.g.*, Δ for $\mathcal{J}^B_{1,*}(\Delta)$), (2) process identifiers, and (3) maybe the number of processes.

A *configuration* of $\mathcal{A}$ for V is a vector of n components $\gamma = (s_1, s_2, \ldots, s_n)$, where s_1 to s_n represent the states of the processes in V. Let γ_1 be the initial configuration of $\mathcal{A}$ for V. For any (synchronous) round $i \geq 1$, the system moves from the current configuration γ_i to some configuration γ_{i+1}, where γ_i (resp. γ_{i+1}) is referred to as the configuration *at the beginning (resp. end) of Round i*. Such a move is atomically performed by every process $p \in V$ according to the following three steps, defined in its local algorithm $\mathcal{A}(p)$: (1) p sends a message consisting of all or a part of its local state in γ_i using the primitive `SEND()`, (2) using Primitive `RECEIVE()`, p receives all messages sent by processes in $\mathcal{IN}(p)^i$, and (3) p computes its state in γ_{i+1}. An *execution* of a distributed algorithm $\mathcal{A}$ in the dynamic graph $\mathcal{G} = G_1, G_2, \ldots$ is an infinite sequence of configurations $\gamma_1, \gamma_2, \ldots$ of $\mathcal{A}$ for V such that $\forall i > 0$, γ_{i+1} is obtained by executing a synchronous round of $\mathcal{A}$ on γ_i based on the communication network at Round i, *i.e.*, the graph G_i.

2.3 Stabilizing Leader Election in Recurring DG Classes

Research on self-stabilization in highly dynamic identified message-passing systems has been initiated in [2]. Notably, [2] adapted the definition of self-stabilization to handle recurring DG classes. We recall this definition below. Then, we accommodate the definition of pseudo-stabilization, that was initially defined in the static context, to recurring DG classes. Finally, we define what we mean by stabilizing leader election.

Stabilization in Recurring DG Classes. Let $\mathcal{A}$ be a distributed algorithm, SP be a specification (*i.e.*, a predicate over configuration sequences), and C be a recurring DG class.

Definition 2.2 (Self-stabilization). An algorithm $\mathcal{A}$ is *self-stabilizing* for SP on C if and only if for every set of processes V, there exists a subset of configurations $\mathcal{L}$ of $\mathcal{A}$ for V, called *legitimate configurations*, such that for every $\mathcal{G} \in C$ with set of processes V,

(1) for every configuration γ of $\mathcal{A}$ for V, every execution of $\mathcal{A}$ in $\mathcal{G}$ starting from γ contains a legitimate configuration $\gamma' \in \mathcal{L}$ (*Convergence*), and
(2) for every legitimate configuration $\gamma \in \mathcal{L}$ and every execution e in $\mathcal{G}$ starting from γ, $SP(e)$ holds (*Correctness*).

The *length of the stabilization phase* of an execution e is the length of its longest prefix containing no legitimate configuration. The

stabilization time in rounds is the maximum length of a stabilization phase over all possible executions.

Basically, a self-stabilizing system, starting from an arbitrary configuration, eventually reaches a configuration from which it *cannot* deviate from its intended specification. In the context of static networks, Burns *et al.* have defined [8] a weak variant of self-stabilization called *pseudo-stabilization*. Intuitively, regardless the initial configuration of the system, a pseudo-stabilizing system eventually reaches a configuration from which it *does not* deviate from its intended specification. In more detail, self-stabilization requires the system to converge to a (legitimate) configuration from which all possible execution suffixes satisfy the intended specification, regardless the initial configuration and despite the adversarial behavior of the environment. In contrast, pseudo-stabilization just imposes every execution (still starting from any configuration) to have a correct suffix, *i.e.*, a suffix satisfying the intended specification. Hence, there may exist executions whose correct suffix is only constituted of configurations for which there always exists a possible future where the system deviates for its specification. As a consequence, we cannot in general define a set of legitimate configurations for pseudo-stabilizing solutions: this notion is irrelevant in pseudo-stabilization. In other words, in pseudo-stabilization, the fact that the system does deviate from its specification in a given correct suffix is sometimes due to the "friendly" behavior of the environment during that suffix.

The difference between self-stabilization and pseudo-stabilization may seem to be very tight, but is actually fundamental. Indeed, in [8], pseudo-stabilization has been shown to be strictly weaker than self-stabilization in terms of expressive power. That is, there are systems and problems for which pseudo-stabilizing solutions do exist but self-stabilizing ones do not. As a matter of facts, they show that the data-link problem can be pseudo-stabilizingly, yet not self-stabilizingly, solved in static message passing systems where the link capacity is unbounded and the process memories are finite.

Below, we give a definition of pseudo-stabilization for the context of recurring DG classes.

Definition 2.3 (Pseudo-stabilization). $\mathcal{A}$ is *pseudo-stabilizing* for SP on $\mathcal{C}$ if and only if for every set of processes V, every $\mathcal{G} \in \mathcal{C}$ with set of processes V, and every configuration γ of $\mathcal{A}$ for V, every execution of $\mathcal{A}$ in $\mathcal{G}$ starting from γ contains a suffix satisfying SP.

The *length of the pseudo-stabilization phase* of an execution $\gamma_1, \gamma_2, \ldots$ is the minimum index i such that $SP(\gamma_{i+1}, \gamma_{i+2}, \ldots)$ holds. The *pseudo-stabilization time* in rounds is the maximum length of a pseudo-stabilization phase over all possible executions.

Remark 2.1. *By definition, if $\mathcal{A}$ is* self-stabilizing *for SP on $\mathcal{C}$, then $\mathcal{A}$ is also* pseudo-stabilizing *for SP on $\mathcal{C}$ (but the reverse is not necessarily true).*

Moreover, the length of the pseudo-stabilization phase is less than or equal to that of the stabilization phase in a given execution of $\mathcal{A}$; and so are its the pseudo-stabilization and stabilization time.

Stabilizing Leader Election. The *leader election* problem consists in distinguishing a single process in the system. In identified networks, the election usually consists in making the processes agree on one of the identifiers held by processes. The identifier of the elected process is then stored at each process p as an output variable, denoted here by $lid(p)$.

In the following, we call *fake ID* any value $v \in IDSET$ (recall that $IDSET$ is the definition domain of the identifiers) such that v is not assigned as a process identifier in the system, *i.e.*, there is no process $p \in V$ such that $id(p) = v$. In the stabilizing context, the output variables lid may be initially corrupted; in particular some of them may be initially assigned to *fake IDs*. Despite such fake IDs, the goal of a self-stabilizing algorithm is to make the system converge to a legitimate configuration γ from which a unique process is forever adopted as leader by all processes whatever be the execution suffix, *i.e.*, $\exists p \in V$ such that $\forall q \in V, lid(q) = id(p)$ forever in all possible execution suffixes starting from γ. The goal of pseudo-stabilizing algorithm is to ensure the existence of an execution suffix where a unique process is forever adopted as leader by all processes. Hence, the *leader election specification* SP_{LE} can be formulated as follows: a sequence of configurations $\gamma_1, \gamma_2, \ldots$ satisfies SP_{LE} if and only if $\exists p \in V$ such that $\forall i \geq 1, \forall q \in V$, the value of $lid(q)$ in Configuration γ_i is $id(p)$.

In the sequel, we say that an algorithm is a *self- (resp. pseudo-) stabilizing leader election algorithm* for the recurring DG class $\mathcal{C}$ if and only if it is self- (resp. pseudo-) stabilizing for SP_{LE} on $\mathcal{C}$.

3 IMPOSSIBILITY RESULTS

In this section, we first show impossibility results on self-stabilizing leader election in DG classes with a source. Then, we show that even pseudo-stabilization is impossible for the leader election problem in $\mathcal{J}^Q_{1,*}(\Delta)$, $\mathcal{J}_{1,*}$, and DG classes with a sink. Finally, we consider the time complexity issue in $\mathcal{J}^B_{1,*}(\Delta)$: we show that the pseudo-stabilization time cannot be bounded in this class. This latter result applies, in particular, to Algorithm $\mathcal{LE}$ presented in Section 4.

All results of this section are based on the notion of *indistinguishable* executions [13]. The principle of this proof scheme is as follows. First, we assume, by the contradiction, that a deterministic leader election algorithm has a given property (related to pseudo- or self- stabilization) in a particular DG class. Then, we consider two sets of processes of same cardinality that differ by only few processes (in the usual case, they differ by only one process). We exhibit two particular executions, one for each set, on two well-chosen DGs of the class. The stabilizing election succeeds in the first one. Moreover, all or a part of processes common to both sets behave exactly the same in both executions: we chose the DG and the initial configuration of the second execution in such way that these processes start with the same local states and receive the sames messages at the same times in both executions; justifying then the term "indistinguishable". Now, the behavior of those processes make the stabilizing election fail in the second execution, leading to a contradiction.

On self-stabilizing leader election in DG classes with a source. We first show that no deterministic algorithm can solve the self-stabilizing leader election in DG classes with a source. To that goal, we first introduce some DGs and a helpful intermediate result (see Lemma 3.2) that will be used in several impossibility proofs. Then, we prove the result for Classes $\mathcal{J}^B_{1,*}(\Delta)$, $\mathcal{J}^Q_{1,*}(\Delta)$, and $\mathcal{J}_{1,*}$.

Definition 3.1. Let $PK(X, y)$ be the digraph with vertex set X and edge set $\{(p, q) : p \in X \setminus \{y\} \wedge q \in X \wedge p \neq q\}$. $PK(X, y)$ is "quasi-complete" in the sense that only links outgoing from y are missing. Let $\mathcal{PK}(X, y)$ be the dynamic graph $G_1, G_2, \ldots$ such that $|X| \geq 2$, $y \in X$, and $\forall i \geq 1, G_i = PK(X, y)$.

Remark 3.2. *By definition, every dynamic graph $\mathcal{PK}(X, y)$ belongs to $\mathcal{J}^B_{1,*}(\Delta)$, for every $\Delta \in \mathbb{N}^*$. Indeed, all processes, except y, are timely sources that can always reach all other processes within one round. In contrast, y can never transmit information to any other process.*

The following lemma formally asserts that in $\mathcal{PK}(V, p)$, Process p cannot be finally elected since the other processes have no mean to detect that $id(p)$ is not a fake ID. Indeed, by definition of $\mathcal{PK}(V, p)$, they never receive any message from p.

Lemma 3.2. *Let V be a set of at least two processes. Let $p \in V$ and $\Delta \in \mathbb{N}^*$. Let $\mathcal{A}$ be any deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$. Let γ_1 be any configuration of $\mathcal{A}$ for V where $\forall q \in V$, $lid(q) = id(p)$.*

Then, in the execution $e = \gamma_1, \gamma_2, \ldots$ of $\mathcal{A}$ in $\mathcal{PK}(V, p)$, at least one process eventually modifies the value of its lid variable.

Proof. Assume, by the contradiction, that $\forall i \in \mathbb{N}^*, \forall q \in V$, we have $lid(q) = id(p)$ in γ_i.

Let v be any process such that $v \notin V$ (in particular, $id(v) \neq id(q), \forall q \in V$). Let $V' = V \setminus \{p\} \cup \{v\}$ and γ'_1 be any configuration of $\mathcal{A}$ for V' such that

(1) v has any local state in γ'_1, and
(2) $\forall q \in V' \setminus \{v\}$, the local state of q is the same in γ'_1 and γ_1.

The only difference between γ_1 and γ'_1 is that p has been replaced by v (with an arbitrary local state). We now consider the execution $e' = \gamma'_1, \gamma'_2, \ldots$ of $\mathcal{A}$ in $\mathcal{PK}(V', v)$ (recall that $\mathcal{PK}(V', v) \in \mathcal{J}^B_{1,*}(\Delta)$; see Remark 3.2).

Claim 3.2.*: $\forall q \in V' \setminus \{v\}, \forall i \in \mathbb{N}^*$, *$q$ has the same local state in γ'_i and γ_i.*

Proof of the claim: By induction on i. The base case $i = 1$ is trivial, by construction of γ'_1. Let $i \geq 1$. By induction hypothesis, $\forall q \in V' \setminus \{v\}$, q has the same local state in γ'_i and γ_i. Let x be any process in $V' \setminus \{v\}$. Since by construction $V' \setminus \{v\} = V \setminus \{p\}$, the in-neighborhood of x is $V' \setminus \{v\}$ at the beginning of Round i both in e and e'. So, x receives the same set of messages and takes the same state in Round i of both e and e', since $\mathcal{A}$ is deterministic. Hence, $\forall q \in V' \setminus \{v\}$, q has the same local state in γ'_{i+1} and γ_{i+1}.

By Claim 3.2.*, $\forall q \in V' \setminus \{v\}$, $lid(q)$ is constantly equal to $id(p)$ in e'. Now, $p \notin V'$ (in other words, $id(p)$ is a fake ID for V'). So, e' has no suffix satisfying SP_{LE}: $\mathcal{A}$ is not a pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$, a contradiction. □

The next theorem is a direct consequence of Lemma 3.2.

Theorem 3.3. *Let $\Delta \in \mathbb{N}^*$. There is no deterministic self-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$.*

Proof. We proceed by contradiction. Let $\mathcal{A}$ be a deterministic self-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$. Let V be any set of at least two processes. Let γ_1 be any legitimate configuration of $\mathcal{A}$ for $\mathcal{J}^B_{1,*}(\Delta)$. Let $\ell \in V$ be the elected leader in γ_1, *i.e.*, $\forall q \in V$, $lid(q) = id(\ell)$ in γ_1. Consider now the execution $e = \gamma_1, \gamma_2, \ldots$ of $\mathcal{A}$ in $\mathcal{PK}(V, \ell)$ (recall that $\mathcal{PK}(V, \ell) \in \mathcal{J}^B_{1,*}(\Delta)$; see Remark 3.2). By Remark 2.1, Lemma 3.2 applies: there exists a process $p \in V$ and a round $i \in \mathbb{N}^*$, such that $lid(p) = id(\ell)$ at the beginning of Round i, but $lid(p) \neq id(\ell)$ at the end of Round i. Hence, $SP_{LE}(e)$ does not hold, a contradiction to the correctness property of Definition 2.2. □

By inclusion (see Figure 2.1) and Theorem 2.1, this impossibility immediately extends to other classes with a source, *i.e.*, $\mathcal{J}^Q_{1,*}(\Delta)$ and $\mathcal{J}_{1,*}$.

Corollary 3.4. *Let $\Delta \in \mathbb{N}^*$. There is no deterministic self-stabilizing leader election algorithm neither for $\mathcal{J}^Q_{1,*}(\Delta)$ nor $\mathcal{J}_{1,*}$.*

On pseudo-stabilizing leader election in $\mathcal{J}^Q_{1,*}(\Delta)$ and $\mathcal{J}_{1,*}$. We now show that no deterministic algorithm can solve the pseudo-stabilizing leader election in Classes $\mathcal{J}^Q_{1,*}(\Delta)$ and $\mathcal{J}_{1,*}$. We first prove the result for Class $\mathcal{J}^Q_{1,*}(\Delta)$ (see Theorem 3.5) and then extend it to $\mathcal{J}_{1,*}$ as a direct consequence of Theorem 2.1 and Figure 2.1 (see Corollary 3.6). Our result is tight since we will propose a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$ in Section 4.

The result is obtained by contradiction: we consider a set of at least two processes and construct on the fly an execution together with its associated DG of $\mathcal{J}^Q_{1,*}(\Delta)$ in such a way that the leader election fails. We start in a configuration where no leader is elected and let the pseudo-stabilizing leader election algorithm execute in a complete graph until a leader is elected (it will happen since the algorithm is assumed to be pseudo-stabilizing). Once a leader, say ℓ, is elected, we disturb the network by making the execution continue in $\mathcal{PK}(V, \ell)$ until a process changes its leader (since $\mathcal{J}^B_{1,*}(\Delta) \subset \mathcal{J}^Q_{1,*}(\Delta)$, Lemma 3.2 applies) and then switch back to a complete graph, and so on. Hence, there is never a permanent leader in the execution. Moreover, since the constructed graph contains infinitely many complete graphs, it trivially belongs to $\mathcal{J}^Q_{1,*}(\Delta)$, for every $\Delta \in \mathbb{N}^*$.

Theorem 3.5. *Let $\Delta \in \mathbb{N}^*$. There is no deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^Q_{1,*}(\Delta)$.*

Proof. We proceed by the contradiction. Let $\mathcal{A}$ be a pseudo-stabilizing leader election algorithm for $\mathcal{J}^Q_{1,*}(\Delta)$. Let V be a set of at least two processes.

Claim 3.5.*: *Let γ_1 be a configuration of $\mathcal{A}$ for V where there is a unique leader $\ell \in V$. Let $e = \gamma_1, \ldots$ the execution e of $\mathcal{A}$ starting from γ_1 in $\mathcal{PK}(V, \ell)$. Then, there exists $i \in \mathbb{N}^*$ such that ℓ is not the leader in γ_i.*

Proof of the claim: Since $\mathcal{J}^B_{1,*}(\Delta) \subset \mathcal{J}^Q_{1,*}(\Delta)$, the claim is immediate from Lemma 3.2.

Let $\mathcal{G} = G_1, G_2, \ldots$ be a dynamic graph with vertex set V and $e = \gamma_1, \gamma_2, \ldots$ be the execution of $\mathcal{A}$ in $\mathcal{G}$ starting from γ_1 constructed as follows: γ_1 is a configuration where there is no unique leader and $G_1 = K(V)$; then, $\forall i \in \mathbb{N}^*$,

(1) if there is one and the same leader ℓ in both γ_i and the configuration γ_{i+1} computed from γ_i and G_i, then $G_{i+1} = PK(V, \ell)$;

(2) otherwise, $G_{i+1} = K(V)$.

By construction and Claim 3.5.*, e has no suffix satisfying SP_{LE}. Moreover, by Claim 3.5.* again, $\mathcal{G}$ contains $K(V)$ infinitely many times. Hence, $\mathcal{G} \in \mathcal{J}^{Q}_{1,*}(\Delta)$. Consequently, $\mathcal{A}$ is not a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{Q}_{1,*}(\Delta)$, a contradiction. □

Corollary 3.6. *There is no deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}_{1,*}$.*

Overall the previous impossibility result shows that to elect a stable leader, other processes should regularly receive information from that leader. Otherwise, they eventually suspect its identifier to be a fake one. Now, in $\mathcal{J}^{Q}_{1,*}(\Delta)$ and $\mathcal{J}_{1,*}$, no process has any timeliness guarantee on the messages it broadcasts. In contrast, in $\mathcal{J}^{B}_{1,*}(\Delta)$, there is at least one process (a timely source) which is always able to broadcast information within bounded time, making then the pseudo-stabilizing leader election possible.

On pseudo-stabilizing leader election algorithm in classes with a sink. We now introduce some DGs allowing us to show that no pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{*,1}(\Delta)$ exists (see Theorem 3.8). Then, we extend this result to every classes with a sink and self-stabilizing algorithms, as direct consequences of Theorem 2.1, Figure 2.1, and Remark 2.1 (see Corollary 3.9).

Definition 3.7. Let X be any vertex set such that $|X| \geq 2$. For every $y \in X$, let $S(X, y)$ be the digraph with vertex set X, $y \in X$, and edge set $\{(p, y) : p \in X \setminus \{y\}\}$. $S(X, y)$ is a "sink star" in the sense that y has no outgoing link and each vertex $p \neq y$ has only one outgoing link to y. Let $\mathcal{S}(X, y)$ be the dynamic graph $G_1, G_2, \ldots$ such that $\forall i \geq 1, G_i = S(X, y)$.

Remark 3.3. *By definition, every dynamic graph $\mathcal{S}(X, y)$ belongs to $\mathcal{J}^{B}_{*,1}(\Delta)$, for every $\Delta \in \mathbb{N}^*$. Indeed, y is a timely sink that can always be reached from all other processes in one round. However, y can never transmit information to any other process.*

The overall idea of this result is the following: consider a set V of at least three processes, one process p of V, and the DG $\mathcal{S}(V, p)$; then every leaf of the star (at least two) will eventually choose itself as leader since it has no mean to guess any identifier of some other process. Indeed, using $\mathcal{S}(V, p)$, no process, except p, can receive messages.

Theorem 3.8. *Let $\Delta \in \mathbb{N}^*$. There is no deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{*,1}(\Delta)$.*

Proof. We proceed by the contradiction. Let V be a set of at least three processes and $p \in V$. Let $\mathcal{A}$ be a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{*,1}(\Delta)$. Let $e = \gamma_1, \gamma_2, \ldots$ be any execution of $\mathcal{A}$ in $\mathcal{S}(V, p)$.

To obtain the contradiction, we show below that for every process $q \in V \setminus \{p\}$, eventually $lid(q) = id(q)$ forever in e. Since $|V \setminus \{p\}| \geq 2$, we immediately obtain the contradiction.

Let $q \in V \setminus \{p\}$. Since $\mathcal{A}$ is a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{*,1}(\Delta)$, there exist $i \in \mathbb{N}^*$ and $v \in V$, such that $\forall j \geq i, lid(q) = id(v)$ in γ_j. Assume, by the contradiction, that $v \neq q$. Let w be any process such that $w \notin V$ (in particular, $id(w) \neq id(x), \forall x \in V$). Let $V' = V \setminus \{v\} \cup \{w\}$. Let γ'_1 be any configuration of $\mathcal{A}$ for V' such that

(1) w has any local state in γ'_1, and
(2) $\forall x \in V' \setminus \{w\}$, the local state of x is the same in γ'_1 and γ_i.

The only difference between γ_i and γ'_1 is that v has been replaced by w (with an arbitrary local state). We now consider the execution $e' = \gamma'_1, \gamma'_2, \ldots$ of $\mathcal{A}$ in $\mathcal{S}(V', w)$ (recall that $\mathcal{S}(V', w) \in \mathcal{J}^{B}_{*,1}(\Delta)$; see Remark 3.3).

Claim 3.8.*: *$\forall j \in \mathbb{N}^*$, the local state of q is the same in γ'_j and γ_{i+j-1}.*

Proof of the claim: By induction on j. The base case $j = 1$ is trivial, by construction of γ'_1. Let $j \geq 1$. By induction hypothesis, $\gamma'_j(q) = \gamma_{i+j-1}(q)$. By construction, the in-neighborhood of q is empty at the beginning of both Round $i + j - 1$ in e and Round j in e'. So, q receives no message and takes the same state in both Round $i + j - 1$ of e and Round j of e', since $\mathcal{A}$ is deterministic. Hence, the local state of q is the same in γ'_{j+1} and γ_{i+j}.

By Claim 3.8.*, $lid(q) = id(v)$ forever in e'. Now, $id(v) \notin V'$ (in other words, $id(v)$ is a fake ID for V'). So, e' has no suffix satisfying SP_{LE}: $\mathcal{A}$ is not a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{*,1}(\Delta)$, a contradiction. □

Corollary 3.9. *Let $\Delta \in \mathbb{N}^*$. There is no deterministic pseudo-stabilizing leader election algorithm neither for $\mathcal{J}^{Q}_{*,1}(\Delta)$ nor $\mathcal{J}_{*,1}$. There is no deterministic self-stabilizing leader election algorithm neither for $\mathcal{J}^{B}_{*,1}(\Delta)$, $\mathcal{J}^{Q}_{*,1}(\Delta)$, nor $\mathcal{J}_{*,1}$.*

On time complexity of deterministic pseudo-stabilizing leader election in $\mathcal{J}^{B}_{1,*}(\Delta)$. Below, we show that the length of the pseudo-stabilization phase of any deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{1,*}(\Delta)$ cannot be bounded. We first define some DG used in the proof.

Definition 3.10. Let $K(X) = (X, E)$ be the complete digraph, *i.e.*, $E = \{(p, q) : p, q \in X \wedge p \neq q\}$. Let $\mathcal{K}(X)$ be the dynamic graph $G_1, G_2, \ldots$ such that $\forall i \geq 1, G_i = K(X)$.

Theorem 3.11 is still shown by contradiction. Consider an execution which begins in an arbitrary configuration while the network is complete. Assume the network remains complete until the end of Round $f(n, \Delta)$. By hypothesis, a leader, say ℓ, is elected at that time. Then, we disturb the system by making the execution continue in the graph $\mathcal{PK}(V, \ell)$. Now, in one hand, we know (from Lemma 3.2) that a process will eventually change its leader, falsifying the specification after Round $f(n, \Delta)$; and in the other hand any DG starting with a finite number of complete graphs followed by $\mathcal{PK}(V, \ell)$ belong to $\mathcal{J}^{B}_{1,*}(\Delta)$. Hence, we obtain the contradiction.

Theorem 3.11. *Let $\Delta \in \mathbb{N}^*$ and $n \geq 2$. Let $\mathcal{A}$ be a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^{B}_{1,*}(\Delta)$. There exists no function $f : \mathbb{N}^* \times \mathbb{N}^* \to \mathbb{N}$ such that $\forall \mathcal{G} \in \mathcal{J}^{B}_{1,*}(\Delta)$ with a vertex set of n processes, the length of the pseudo-stabilization phase of every execution of $\mathcal{A}$ in $\mathcal{G}$ is less than or equal to $f(n, \Delta)$.*

Proof. Assume by the contradiction that such a function f exists. By definition, $\mathcal{K}(V) \in \mathcal{J}^{B}_{1,*}(\Delta)$. Let $\gamma_1, \gamma_2, \ldots$ be any execution of $\mathcal{A}$ in $\mathcal{K}(V)$. Let $i = f(n, \Delta) + 1$. By hypothesis, there exists $\ell \in V$ such that $\forall p \in V, \gamma_i(p).lid = id(\ell)$.

Let e be the execution of $\mathcal{A}$ in $\mathcal{PK}(V,\ell)$ starting from γ_i. By Lemma 3.2, at least one process eventually modifies the value of its lid variable in e (this in particular means that $SP_{LE}(e)$ does not hold).

Now, $\mathcal{G} = (K(V))^{i-1}, \mathcal{PK}(V,\ell)$, *i.e.*, the sequence made of $i-1$ graphs $K(V)$ followed by the sequence of graphs in $\mathcal{PK}(V,\ell)$, is a dynamic graph with vertex set V that belongs to $\mathcal{J}^B_{1,*}(\Delta)$. Moreover, $e' = \gamma_1, \gamma_2, ..., \gamma_{i-1}, e$, *i.e.*, the sequence made of $\gamma_1, \gamma_2, ..., \gamma_{i-1}$ followed the sequence of configurations in e, is an execution of $\mathcal{A}$ in $\mathcal{G}$ where $SP_{LE}(e)$ does not hold. Hence, the length of the pseudo-stabilization phase in e' is greater than $f(n,\Delta)$, a contradiction. □

4 PSEUDO-STABILIZING LEADER ELECTION IN $\mathcal{J}^B_{1,*}(\Delta)$

Algorithm $\mathcal{LE}$, whose code is given in Algorithm 1, is a pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$. Its goal is to elect what we call a *stable* process. Intuitively, a process p is stable if eventually all other processes receive (maybe indirectly) information about p at least every Δ rounds.

Basically, in the algorithm all processes initiate a broadcast at each round. So, since there exist timely sources in Class $\mathcal{J}^B_{1,*}(\Delta)$, there are stable processes in the system, namely the timely sources themselves.

To evaluate the stability of a process p, we use a so-called *counter of suspicion*. The suspicion counter of p is maintained by p itself and is incremented each time p learns that some other process does not consider it as stable. In the following, we call *suspicion value* of p the current value of the suspicion counter of p.

Due to initial inconsistencies, a suspicion counter may be reset once during the first round. Yet, after the first round, each counter is monotonically nondecreasing. Moreover, the counter of each timely source is eventually constant since it is stable, by definition. Finally, each process aims at electing a process with the minimum suspicion value (we use identifiers to break ties) since such a process will be a stable process.

To implement this principle, each process p maintains four variables: $lid(p)$, $msgs(p)$, $Lstable(p)$, and $Gstable(p)$. Variable $lid(p)$ is the output of the algorithm, it eventually contains the identifier of the elected process. The three other variables are more complex data structures. Basically, Process p stores in $msgs(p)$ the messages that it will send at the beginning of the next round; see Line 2. $Lstable(p)$ and $Gstable(p)$ are used to detect stable processes. They are both of type $MapType$.

Type $MapType$**.** The main objective of Algorithm $\mathcal{LE}$ is to identify a set of stable processes. To achieve this, processes exchange information about each process they heard about. This information is stored in data structures of type $MapType$, *i.e.*, *maps* containing tuples of three elements: an identifier $id \in IDSET$, a natural number $susp$, representing the (maybe outdated) suspicion value of the process identified by id, and a timer value $ttl \in \{0,\ldots,\Delta\}$ (ttl stands for "time to live").

A map of type $MapType$ is indexed by the first element id of each tuple it contains. So, there exists at most one tuple $\langle id,-,-\rangle$ in any variable of type $MapType$. We simply denote by $M[id]$ the tuple of index id in the map M. Moreover, for sake of simplicity, we write $id \in M$ to mean that the tuple $M[id]$ exists, *i.e.*, M contains a tuple of index id: $\langle id,-,-\rangle \in M$. Of course, insertion and removal functions maintain the uniqueness of each index in the map. In particular, if a tuple $\langle id,s,t\rangle$ is inserted into the map M while $M[id]$ already exists, $M[id]$ is just refreshed with the new values s and t.

Locally Stable Processes. Every process p stores into its $Lstable(p)$ variable information — precisely, an identifier, a suspicion value, and a timer value — for each process that is *locally stable* at p, *i.e.*, for each process q that p currently considers as a candidate for the leader election because it receives information from q at most Δ rounds ago.

Globally Stable Processes. For every process p, its $Gstable(p)$ variable contains information — *i.e.*, tuples made of an identifier, a suspicion value, and a timer value — related to *globally stable processes*, meaning processes that are *locally stable* at any process (and so not only at p). Eventually all stable processes (in particular, the timely sources) should forever belong to $Gstable(p)$, for every process p. Moreover, at each round every process selects the value of its output $lid(p)$ among identifiers stored into $Gstable(p)$. Precisely, p chooses an ID in $Gstable(p)$ associated with the minimum suspicion value (we use IDs to break ties); see Line 27 and Macro $minSusp(p)$.

Messages. Processes exchange information stored into what we call *records*. Actually, a record R is a tuple $\langle id, LSPs, ttl\rangle$, where $R.id \in IDSET$, $R.LSPs$ is a map of type $MapType$, and $R.ttl \in \{0,\ldots,\Delta\}$ is a timer value. The variable $msgs(p)$ is used to store the records that will be sent by Process p at the beginning of the next round. Actually, $msgs(p)$ is a set (and so not a map) meaning in particular that it may contain several records tagged with the same identifier.

At each round, each process p initiates the broadcast of a record $\langle id(p), Lstable(p), \Delta\rangle$ by inserting it into $msgs(p)$; see Line 26. Note that in each record $\langle id(p), Lstable(p), \Delta\rangle$ initiated by p, we have $id(p) \in Lstable(p)$ (if necessary, $id(p)$ is added in $Lstable(p)$ at Line 4, and then never removed). Then, the timer in the record is used to relay the record during Δ rounds. The goal of this broadcast is to inform other processes about the current list of processes that are locally stable at p with their associated suspicion value known by p (*n.b.*, those suspicion values may be outdated).

At each round, received records that should be relayed are collected into $msgs(p)$ and their associated timer is decremented; see Lines 13 and 25. Only well-formed records R (*i.e.*, satisfying $R.id \in R.LSPs$) with non-zero timer will be sent during the next round; see Lines 2 and 24. The condition $R.id \in R.LSPs$ allows to eliminate some spurious messages. The timer mechanism, in particular, ensures that records tagged with fake IDs are eventually no more relayed since such records are never initiated.

Management of $Lstable(p)$**.** Recall that $Lstable(p)$ contains the identifiers and known suspicion values of processes that are currently considered by p to be locally stable. First, p always considers itself as a locally stable process. Thanks to Line 4, a tuple $\langle id(p),-,\Delta\rangle$ forever belongs to $Lstable(p)$ after the first round. Then, p supplies $Lstable(p)$ using records it receives from other processes. Upon receiving a record $\langle id, LSPs, ttl\rangle$, p inserts the tuple $\langle id, LSPs[id].susp, ttl\rangle$ into $Lstable(p)$ if either id does not appear in $Lstable(p)$, or ttl is greater than the current timer associated

Algorithm 1: Algorithm $\mathcal{LE}$, code for each process p

Inputs: $\Delta \in \mathbb{N}^*$: upper bound on the temporal distance from a timely source to any other process
$id(p) \in IDSET$: ID of p

Type: $MapType$: map containing tuples of the form $\langle id, susp, ttl\rangle \in IDSET \times \mathbb{N} \times \{0, \ldots, \Delta\}$

Local Variables:
$lid(p) \in IDSET$: ID of the leader
$Lstable(p) \in MapType$: locally stable processes for p
$Gstable(p) \in MapType$: globally stable processes collected by p
$msgs(p)$: set containing records of the form $\langle id, LSPs, ttl\rangle \in IDSET \times MapType \times \{0, \ldots, \Delta\}$

Macro: $minSusp(p) = \min\{Gstable(p)[q].susp : q \in Gstable(p)\}$

```
1:  Repeat Forever
2:      SEND({msg ∈ msgs(p) : msg.ttl > 0 ∧ msg.id ∈ msg.LSPs})
3:      mailbox := RECEIVE()
4:      if ⟨id(p), −, Δ⟩ ∉ Lstable(p) then insert ⟨id(p), 0, Δ⟩ in Lstable(p)
5:      if ⟨id(p), −, Δ⟩ ∉ Gstable(p) ∨ Gstable(p)[id(p)].susp ≠ Lstable(p)[id(p)].susp then
6:          insert Lstable(p)[id(p)] in Gstable(p)
7:      forall id ∈ Lstable(p) do
8:          if id ≠ id(p) ∧ Lstable(p)[id].ttl ≠ 0 then Lstable(p)[id].ttl − −
9:      forall id ∈ Gstable(p) do
10:         if id ≠ id(p) ∧ Gstable(p)[id].ttl ≠ 0 then Gstable(p)[id].ttl − −
11:     forall tuples msg = ⟨id, LSPs, ttl⟩ in a message of mailbox do
12:         if id ≠ id(p) then
13:             if ⟨id, −, ttl⟩ ∉ msgs(p) then insert msg = ⟨id, LSPs, ttl⟩ in msgs(p)
14:             if ⟨id, −, −⟩ ∉ Lstable(p) ∨ (⟨id, −, ttl′⟩ ∈ Lstable(p) with ttl′ < ttl) then
15:                 insert ⟨id, LSPs[id].susp, ttl⟩ in Lstable(p);
16:             forall ⟨id′, susp, −⟩ ∈ LSPs : id′ ≠ id(p) do
17:                 insert ⟨id′, susp, Δ⟩ in Gstable(p)
18:             if id(p) ∉ LSPs then Lstable(p)[id(p)].susp + +; Gstable(p)[id(p)].susp + +;
19:     forall id ∈ Lstable(p) do
20:         if Lstable(p)[id].ttl = 0 then remove the tuple associated to id in Lstable(p)
21:     forall id ∈ Gstable(p) do
22:         if Gstable(p)[id].ttl = 0 then remove the tuple associated to id in Gstable(p)
23:     forall msg ∈ msgs(p) do
24:         if msg.ttl = 0 ∨ msg.id ∉ msg.LSPs then remove msg from msgs(p)
25:         else msg.ttl − −
26:     insert ⟨id(p), Lstable(p), Δ⟩ in msgs(p)
27:     lid(p) := min{id ∈ Gstable(p), Gstable(p)[id].susp = minSusp(p)}
```

to id in $Lstable(p)$; see Lines 14-15. Notice that $LSPs[id].susp$ is meant to be the suspicion value of the initiator at the time it started to broadcast the record.

The timer values of each record in $Lstable(p)$ is decremented at each round; see Lines 7-8. Once a timer value has reached 0, the corresponding record is removed from $Lstable(p)$; see Lines 19-20. Hence, an identifier id different from $id(p)$ remains in $Lstable(p)$ only if p receives at least every Δ rounds a record $\langle id, -, -\rangle$ initiated by the process whose ID is id. Notice that, since records tagged with fake IDs are eventually never more received, they also eventually vanish from all $Lstable$ maps.

Management of $Gstable(p)$. Similar to $Lstable(p)$, p always considers itself as a globally stable process. Thank to Line 5, a tuple $\langle id(p), -, \Delta\rangle$ forever belongs to $Gstable(p)$ after the first round. Then, p supplies $Gstable(p)$ with the map $R.LSPs$ attached to records R it receives from other processes. At each round, each identifier different from $id(p)$ present in a map of some received record is meant to be the identifier of a locally stable process at some other process. So, it is meant to identify a globally stable process. Such an identifier and its associated suspicion value is then stored into $Gstable(p)$ with a timer initialized to Δ; see Lines 17-18.

Similar to $Lstable(p)$, the timer values of each record in $Gstable(p)$ is decremented at each round; see Lines 9-10. Moreover, every record whose timer value reaches 0 is removed from $Gstable(p)$; see Lines 21-22. Hence, an identifier id different from $id(p)$ remains in $Gstable(p)$ only if p receives at least every Δ rounds a record from some other process notifying that a process identified by id is one of its locally stable processes. Remark that, since fake IDs eventually vanish from all $Lstable$ maps, fake IDs also eventually vanish from all $Gstable$ maps.

Suspicion value. To simplify a bit the algorithm design, the suspicion value of each process p is stored in both $Lstable(p)[id(p)].susp$ and $Gstable(p)[id(p)].susp$. So, the values $Lstable(p)[id(p)].susp$ and $Gstable(p)[id(p)].susp$ should be kept equal; see Lines 5, 6, and 18. Moreover, after the first round, the suspicion value of p is monotonically nondecreasing.

Actually, p increments its suspicion value each time it realizes that it is not part of the locally stable processes of some other process q, *i.e.*, each time it receives a record R initiated by q such that $id(p) \notin R.LSPs$; see Line 18.

Using the broadcasts it initiates at each round, p aims at continuously propagating its latest suspicion value to all other processes.

Fake IDs. We have seen that records tagged with fake IDs are eventually no more sent. Furthermore, by a domino effect, fake IDs are eventually removed from all *Lstable* and *Gstable* maps. Actually, we can prove that no fake ID exists in the system after at most 4Δ rounds.

Pseudo-stabilization. We have seen that every process p collects in $Lstable(p)$ the list of processes that it currently considers to be locally stable, including itself. In particular, an identifier id different from $id(p)$ remains in $Lstable(p)$ only if p received at least every Δ rounds a record $\langle id, -, - \rangle$ initiated by the process identified by id. Hence, if the temporal distance between two processes p and q is always at most Δ, then eventually $id(q) \in Lstable(p)$ forever. A direct consequence of this property is that the identifier of every timely source eventually constantly belongs to the *Lstable* map of every process. Actually, we can show that the identifier of each timely source forever belongs to any *Lstable* map after at most $2\Delta + 1$ rounds.

Gstable maps are managed similarly to *Lstable* maps using timers. Each process p forever appears in its own *Gstable* map after at most one round. Then, p tries to fill it with information (including the suspicion value) about the local stable processes at all other processes.

Again, since p receives records initiated by each timely source at least every Δ rounds, almost up to date information about the processes locally stable at each timely source are eventually always present in $Gstable(p)$. Notably, identifiers that are eventually forever present in *Lstable* maps of all timely sources, in particular their own identifiers, are eventually forever present in each *Gstable* map (actually, after at most $3\Delta + 2$ rounds).

Recall that after the first round, the suspicion value of every process is monotonically nondecreasing. Moreover, the suspicion value of a process p is incremented only when p receives a record with a list of locally stable processes that does not include its own identifier. Hence, from the previous discussion, we know that each timely source only increments its suspicion value a finite number of times. Moreover, eventually all processes store forever in their *Gstable* map both the identifiers and the final suspicion values of all timely sources.

Consider now a process q whose identifier is infinitely often absent from $Gstable(p)$, for some process p. In this case, there are infinitely many period of Δ rounds where p does not receive any record including $id(q)$ into its list of locally stable processes. This means, in particular, that every timely source broadcasts infinitely many records that do not include $id(q)$ into the attached list of locally stable processes. Since initiated by timely sources, such records reach q infinitely often, and consequently the suspicion counter of q grows infinitely often. Moreover, from the previous discussion, if q is also inserted infinitely often in some *Gstable* map, then its suspicion value in the map grows infinitely often.

Overall, the *Gstable* map of every process eventually forever contains the identifiers and final suspicion values of all processes (including timely sources) whose suspicion value is eventually constant. Moreover, every other process which regularly appears in the map do so with an associated suspicion value that increases again and again. Now, at the end of each round, each process p selects an identifier in its *Gstable* map with the minimum attached suspicion value (the order on identifiers may be used to break ties). Hence, all processes eventually forever elect the same process.

Speculation. The pseudo-stabilization time of Algorithm $\mathcal{LE}$ cannot be bounded in $\mathcal{J}^B_{1,*}(\Delta)$ by Theorem 3.11. However, Algorithm $\mathcal{LE}$ is speculative in the sense that its pseudo-stabilization time in $\mathcal{J}^B_{*,*}(\Delta) \subset \mathcal{J}^B_{1,*}(\Delta)$ is bounded. This result is due to the fact that all processes are timely sources in $\mathcal{J}^B_{*,*}(\Delta)$.

In more detail, the suspicion value of each process becomes constant after executing at most $2\Delta + 1$ rounds in a DG of $\mathcal{J}^B_{*,*}(\Delta)$. Then, we can show that during the next 4Δ rounds, all fake IDs vanish and all nodes are inserted forever into all *Gstable* maps with their final suspicion values. Hence, at the beginning of the next round (*i.e.*, after at most $6\Delta + 2$ rounds), a unique leader is forever elected by all processes.

Overall, we obtain the following theorem. Its detailed proof is available in the full paper online.

Theorem 4.1. *Algorithm $\mathcal{LE}$ is a deterministic pseudo-stabilizing leader election algorithm for $\mathcal{J}^B_{1,*}(\Delta)$. Moreover, Algorithm $\mathcal{LE}$ is speculative: its pseudo-stabilization time in $\mathcal{J}^B_{*,*}(\Delta)$ ($\subset \mathcal{J}^B_{1,*}(\Delta)$) is at most $6\Delta + 2$ rounds.*

5 CONCLUSION

We have studied the expressive power, w.r.t. the (deterministic) leader election problem, of pseudo- and self-stabilization in nine classes of dynamic graphs. Our results show that self-stabilizing leader election can only be solved in the three classes studied in [2]. Furthermore, even pseudo-stabilizing leader election cannot be solved in the remaining classes, except in the class where at least one process is a timely source. We have defined those classes by analogy with classes mainly studied for the Ω problem in crash-prone partially synchronous systems. We have simply skipped several dynamic patterns classically used in those systems, *e.g.*, patterns related to eventual timeliness and the notion of bi-source. Actually, as explained in [13], the fact that the bound immediately holds (timeliness) or only eventually (eventual timeliness) has no impact on stabilizing systems: just consider the first configuration from which the bound is guaranteed as the initial point of observation. Similarly, dealing with dynamic graphs where at least one process is a bi-source (*i.e.*, a process which is both a source and a sink) is not an issue in the expressive point of view. Indeed, the existence of a bi-source makes those dynamic graphs belonging to the class $\mathcal{J}_{*,*}$ since any bi-source acts as a hub during flooding. Concerning now the efficiency of solutions, our results give a broad answer in terms of time complexity, in particular about convergence time. However, the space complexity aspect remains widely open. In particular, the stabilizing solution we proposed for $\mathcal{J}^B_{1,*}(\Delta)$ and the one proposed for $\mathcal{J}_{*,*}$ in [2] use an infinite memory. We conjecture that this drawback cannot be precluded.

ACKNOWLEDGMENTS

This study has been partially supported by the French ANR projects ANR-16-CE40-0023 (DESCARTES) and ANR-16 CE25-0009-03 (ESTATE).

REFERENCES

[1] Marcos Kawazoe Aguilera, Carole Delporte-Gallet, Hugues Fauconnier, and Sam Toueg. 2008. On implementing omega in systems with weak reliability and synchrony assumptions. *Distributed Comput.* 21, 4 (2008), 285–314.

[2] Karine Altisen, Stéphane Devismes, Anaïs Durand, Colette Johnen, and Franck Petit. 2021. Self-stabilizing Systems in Spite of High Dynamics. In *ICDCN '21: International Conference on Distributed Computing and Networking*. ACM, Virtual Event, Nara, Japan, 156–165.

[3] Karine Altisen, Stéphane Devismes, Anaïs Durand, and Franck Petit. 2019. Gradual stabilization. *J. Parallel Distributed Comput.* 123 (2019), 26–45.

[4] Matthieu Barjon, Arnaud Casteigts, Serge Chaumette, Colette Johnen, and Yessin M. Neggaz. 2019. Maintaining a Distributed Spanning Forest in Highly Dynamic Networks. *Comput. J.* 62, 2 (2019), 231–246.

[5] Marjorie Bournat, Ajoy K. Datta, and Swan Dubois. 2019. Self-stabilizing robots in highly dynamic environments. *Theor. Comput. Sci.* 772 (2019), 88–110.

[6] Nicolas Braud-Santoni, Swan Dubois, Mohamed-Hamza Kaaouachi, and Franck Petit. 2016. The Next 700 Impossibility Results in Time-Varying Graphs. *Int. J. Netw. Comput.* 6, 1 (2016), 27–41.

[7] Binh-Minh Bui-Xuan, Afonso Ferreira, and Aubin Jarry. 2003. Computing Shortest, Fastest, and Foremost Journeys in Dynamic Networks. *Int. J. Found. Comput. Sci.* 14, 2 (2003), 267–285.

[8] James E. Burns, Mohamed G. Gouda, and Raymond E. Miller. 1993. Stabilization and Pseudo-Stabilization. *Distributed Comput.* 7, 1 (1993), 35–42.

[9] Shukai Cai, Taisuke Izumi, and Koichi Wada. 2012. How to Prove Impossibility Under Global Fairness: On Space Complexity of Self-Stabilizing Leader Election on a Population Protocol Model. *Theory Comput. Syst.* 50, 3 (2012), 433–445.

[10] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. 2012. Time-varying graphs and dynamic networks. *Int. J. Parallel Emergent Distributed Syst.* 27, 5 (2012), 387–408.

[11] Bernadette Charron-Bost and Shlomo Moran. 2019. The firing squad problem revisited. *Theor. Comput. Sci.* 793 (2019), 100–112.

[12] Ajoy K. Datta and Lawrence L. Larmore. 2018. Self-Stabilizing Leader Election in Dynamic Networks. *Theory Comput. Syst.* 62, 5 (2018), 977–1047.

[13] Carole Delporte-Gallet, Stéphane Devismes, and Hugues Fauconnier. 2010. Stabilizing leader election in partial synchronous systems with crash failures. *J. Parallel Distributed Comput.* 70, 1 (2010), 45–58.

[14] Edsger W. Dijkstra. 1974. Self-stabilizing Systems in Spite of Distributed Control. *Commun. ACM* 17, 11 (1974), 643–644.

[15] Shlomi Dolev. 1998. Optimal Time Self-Stabilization in Uniform Dynamic Systems. *Parallel Process. Lett.* 8, 1 (1998), 7–18.

[16] Shlomi Dolev, Ariel Hanemann, Elad Michael Schiller, and Shantanu Sharma. 2012. Self-stabilizing End-to-End Communication in (Bounded Capacity, Omitting, Duplicating and non-FIFO) Dynamic Networks - (Extended Abstract). In *Stabilization, Safety, and Security of Distributed Systems - 14th International Symposium, SSS 2012 (Lecture Notes in Computer Science, Vol. 7596)*, Andréa W. Richa and Christian Scheideler (Eds.). Springer, Toronto, Canada, 133–147.

[17] Shlomi Dolev and Ted Herman. 1997. Superstabilizing Protocols for Dynamic Distributed Systems. *Chic. J. Theor. Comput. Sci.* 1997 (1997).

[18] Shlomi Dolev, Amos Israeli, and Shlomo Moran. 1993. Self-Stabilization of Dynamic Systems Assuming Only Read/Write Atomicity. *Distributed Comput.* 7, 1 (1993), 3–16.

[19] Swan Dubois and Rachid Guerraoui. 2013. Introducing speculation in self-stabilization: an application to mutual exclusion. In *ACM Symposium on Principles of Distributed Computing, PODC '13*, Panagiota Fatourou and Gadi Taubenfeld (Eds.). ACM, Montreal, QC, Canada, 290–298.

[20] Ramakrishna Kotla, Lorenzo Alvisi, Michael Dahlin, Allen Clement, and Edmund L. Wong. 2009. Zyzzyva: Speculative Byzantine fault tolerance. *ACM Trans. Comput. Syst.* 27, 4 (2009), 7:1–7:39.